AI-Powered Technical Interview Trainer

Project Problem Statement & Documentation Blueprint

1. Project Overview & Objective

The **AI-Powered Technical Interview Trainer** is an intelligent, autonomous simulation system designed to help job seekers and students practice, evaluate, and master technical interviews.

Traditional mock interview platforms rely on static question banks or generalized chatbots. This project leverages **Python Flask** and **IBM watsonx.ai (Mistral Model)** with prompt engineering and agentic behavior to conduct dynamic, role-based technical interviews with real-time feedback.

2. Core Technical Stack

- **Backend Framework:** Python, Flask (`app.py`, routing, and JSON payload handlers)
- **Frontend Interface:** HTML5, CSS3, JavaScript (Responsive Chat UI)
- **AI Engine:** IBM watsonx.ai SDK using `mistralai/mistral-small-3-1-24b-instruct-2503`
- **Configuration & Deployment:** `app.json`, `requirements.txt`, and GitHub repository setup

3. Architecture Blueprint

The system architecture follows a clean 3-tier structure ensuring modularity and secure API communication:

```
[ Frontend (HTML/JS Chat UI) ]
       |  (HTTP POST / JSON)
       v
[ Backend Server (Python Flask) ]
       |  (IBM Cloud SDK Call)
       v
[ AI Layer (IBM watsonx.ai / Mistral Model) ]
```

- **Frontend:** Captures user responses and renders real-time conversation dynamically.
- **Flask Server:** Acts as the control bridge, routing payloads securely to IBM Cloud.
- **IBM watsonx.ai Layer:** Executes strict system instructions to simulate a professional technical interviewer.

4. Key Features & Novelty

- **Multi-Turn Role Adaptation:** Dynamically changes question difficulty based on prior answers.
- **Hyper-Personalization:** Tailored sessions focusing on specific tech stacks, programming languages, and experience levels.
- **Real-Time Feedback Loop:** Provides targeted explanations and constructive critique instantly.
- **Enterprise AI Governance:** Built on IBM watsonx.ai, ensuring robust security, tracking, and token usage optimization.

5. GitHub Submission Requirements

The public GitHub repository includes the following mandatory files for evaluation:

- `app.json` - Project metadata and configuration file
- `problem_statement.pdf` - Comprehensive project documentation
- `presentation.pptx` - Final slide deck
- `app.py` & Templates - Core application code
